

Real-Time Stock Market Data Processing Pipeline

Om Ashokkumar Patel, Souptik Sinha, Shiva Raghav Rajasekar,
Arya Shetty, Muthu Nageswaran Kalyani Narayanamoorthy, and Anusha Venkatesh
CSP554 Big Data Technologies
Illinois Institute of Technology
Chicago, IL, USA

Abstract—This paper presents the design, implementation, and evaluation of an end-to-end real-time stock market data processing pipeline built on Apache Kafka, Amazon DynamoDB, and Apache Spark (PySpark). The pipeline ingests five years of daily OHLCV (open, high, low, close, volume) data for five major equities from Yahoo Finance, streams it through a Kafka topic at simulated real-time cadence using a Dockerized broker, processes each record in a Kafka consumer to compute derived financial indicators, persists results to a cloud-hosted DynamoDB table, and performs post-ingestion analytics and visualization using pandas and matplotlib. The completed system stores 6,285 records with zero data-quality failures. DynamoDB write latency averages 22.2 ms per individual insert and 1.0 ms per record in bulk batch mode, confirming that the storage layer is capable of handling high-velocity financial data. Anomaly detection, a 7-day rolling moving average, and volume-spike flagging are computed inline during stream consumption. Five visualization charts produced from the stored data confirm that price trends, volatility events, and volume behavior are captured accurately. The results validate the Kafka-to-DynamoDB pipeline pattern recommended in the financial big data literature and demonstrate that a small team can reproduce a production-grade streaming architecture using readily available open-source and managed cloud tools.

Index Terms—Apache Kafka, Amazon DynamoDB, Apache Spark, real-time data streaming, financial analytics, anomaly detection, big data pipeline, stock market

I. INTRODUCTION

Financial markets produce structured data at a pace that traditional batch-processing systems cannot absorb without introducing unacceptable analytical lag. A single equities exchange generates millions of trade and quote events per day, and derived signals such as moving averages, volatility flags, and volume anomalies must be available within seconds of the originating event if they are to be actionable for traders, risk systems, or automated strategies. Prabhakaran, Jaiswal, and Gandhi [1] identify low-latency streaming infrastructure as a non-negotiable prerequisite for real-time financial analytics, and they specifically recommend a hybrid architecture that combines Apache Kafka for high-throughput ingestion with Amazon DynamoDB for predictably performant cloud storage.

This project addresses that requirement by building a complete big data processing pipeline. The implemented system streams historical stock data through Kafka at a controlled rate, applies in-consumer transformations to compute financial metrics, stores the enriched records in DynamoDB, profiles the raw data using Apache Spark and PySpark, and generates analytical visualizations from the stored dataset.

The five equities chosen for the project are Apple Inc. (AAPL), Alphabet Inc. (GOOGL), Microsoft Corporation (MSFT), Tesla Inc. (TSLA), and Amazon.com Inc. (AMZN). These symbols represent a cross-section of sectors and volatility profiles within the technology-heavy large-cap universe and provide a dataset rich enough to surface meaningful anomaly and volume-spike patterns. The data window spans January 2, 2020 to December 30, 2024, a period that includes the COVID-19 market crash, the 2020 to 2021 growth rally, the 2022 rate-driven correction, and the 2023 to 2024 recovery, giving the anomaly detection logic a realistic distribution of extreme events to identify.

The remainder of this paper is structured as follows. Section II reviews the relevant literature. Section III describes the methodology including data sources, architecture decisions, and analytical techniques. Section IV presents the results organized by pipeline phase. Section V discusses the implications of those results, limitations, and comparisons to prior work. Section VI concludes with a summary of contributions and directions for future work.

II. LITERATURE REVIEW

A. Apache Kafka as the Ingestion Layer

Kreps, Narkhede, and Rao [2] present Kafka as a distributed, high-throughput publish-subscribe messaging system designed for large-scale log processing. Kafka organizes data into topics partitioned into ordered, append-only segments replicated across a cluster of brokers. Consumers pull data at their own pace through offset-based addressing, which eliminates the random-access index overhead present in earlier messaging systems and allows throughput to scale linearly with data volume into the multi-terabyte range. Producer benchmarks in the original paper demonstrate 400,000 messages per second at a batch size of 50, substantially exceeding comparable systems at the time of publication. Kafka’s stateless broker design supports simultaneous real-time and offline consumption from the same topic, meaning a live dashboard and a back-testing job can both read from the pipeline without duplicating data. For this project, a single topic partitioned by ticker symbol allows consumers to subscribe to specific instruments and to replay historical records for model validation.

B. The Kafka-Spark-NoSQL Pipeline Pattern

Nazeer et al. [3] validate the Kafka-to-Spark-to-NoSQL pattern by demonstrating a fully operational real-time text

analytics system using Apache Kafka, Apache Spark Streaming, and Apache Cassandra. Although the application domain is Twitter sentiment analysis rather than financial data, the architectural findings transfer directly. A three-node cluster in that study processes 466,700 messages in 10.7 minutes with sub-minute end-to-end latency, a throughput approximately 140 percent faster than a single-machine baseline, demonstrating near-linear horizontal scaling. This result is important for financial pipelines, where volume surges during earnings announcements or macroeconomic shocks can arrive without warning. The present project substitutes DynamoDB for Cassandra to leverage managed auto-scaling and the ticker-plus-date composite key structure, and substitutes financial time-series aggregation and anomaly detection for text classification.

C. Cloud-Based Financial Analytics and Cost Management

Prabhakaran et al. [1] provide the most directly applicable design guidance by benchmarking Kafka against AWS Kinesis on high-volume tick data and evaluating AWS EMR with Spark Streaming as the processing layer. Their findings show that Kafka outperforms Kinesis on raw throughput for dense tick feeds but that Kinesis is preferable when minimizing operational overhead takes priority over cost per message. The paper documents several cost-management strategies including EC2 Spot Instances for non-critical workloads, tiered storage lifecycle policies to migrate aged data from high-performance to lower-cost tiers, and batched cross-region transfers to reduce per-file egress charges. A payment-processor case study demonstrates that auto-scaling clusters running only during traffic spikes reduce infrastructure costs by approximately 40 percent relative to always-on configurations. The hybrid Kafka-plus-DynamoDB architecture recommended in that paper is the direct basis for the architecture implemented in this work.

D. Amazon DynamoDB at Scale

Elhemali et al. [4] document the design and evolution of Amazon DynamoDB, describing a service capable of handling 89.2 million requests per second at consistent single-digit millisecond latency. DynamoDB’s composite key model, using a partition key and a sort key, maps directly onto financial data: the ticker symbol serves as the partition key to co-locate all records for one instrument, and the ISO 8601 date string serves as the sort key to allow efficient range queries over time. The authors address the hot-partition problem, which arises in financial pipelines when a small number of heavily traded securities receive a disproportionate share of write traffic. DynamoDB’s Global Admission Control separates admission decisions from partition-level capacity, absorbing traffic surges at the routing layer rather than at storage. The on-demand capacity mode automatically handles up to twice the previous peak traffic without manual provisioning, which is essential when volume spikes are triggered by exogenous events that cannot be forecast. Multi-replica write-ahead logging archived to Amazon S3, continuous scrubbing of live replicas against

offline reconstructions, and TLA-plus-verified replication protocols provide the durability guarantees required for financial records.

E. Advanced Event-Streaming Design Lessons from Mofka

Dorier et al. [5] present Mofka, a durable event-streaming framework evaluated against Kafka and Redpanda on production supercomputers. The central architectural innovation is a separation of each event into a small structured metadata header and a larger binary data payload, enabling consumers to filter and route on the header alone without deserializing the payload. For a financial pipeline, this translates to a lightweight header containing the symbol, timestamp, exchange identifier, and event type alongside a heavier payload containing order book snapshots or options chains, so routing-only consumers avoid unnecessary deserialization cost. The paper establishes three actionable lessons from benchmarking: memory allocation strategy measurably affects throughput and must be fixed before scaling infrastructure; batch size must be tuned per consumer group rather than set globally because latency-sensitive consumers require small batches while throughput-oriented consumers benefit from larger ones; and broker scaling does not automatically resolve bottlenecks arising from client-side threading, making profiling a prerequisite for infrastructure decisions. The Mofka benchmarks show 0.22 to 1.33 percent workflow overhead compared to 6.31 to 26.09 percent for Kafka under the same conditions on supercomputer workloads, illustrating that observability instrumentation must be designed into the pipeline from the start.

F. Synthesis

The five reviewed works collectively define the technical foundation for the present pipeline. Kreps et al. [2] establish the performance baseline for Kafka as the ingestion layer. Nazeer et al. [3] confirm horizontal scalability of the Kafka-Spark-NoSQL pattern. Prabhakaran et al. [1] apply that pattern to financial analytics and directly recommend the Kafka-plus-DynamoDB hybrid implemented here. Elhemali et al. [4] justify DynamoDB as the storage layer by addressing hot-partition management, on-demand scaling, and durability. Dorier et al. [5] contribute design principles for event structure, batch tuning, and built-in observability. Together these works provide a coherent and well-evidenced basis for every architectural decision made in the project.

III. METHODOLOGY

A. System Architecture Overview

The pipeline is organized into four sequential phases that map to the project milestones established in the proposal. Phase 1 covers data acquisition and profiling. Phase 2 implements the Kafka streaming layer and the DynamoDB consumer. Phase 3 performs analytics and visualization from the stored data. Phase 4 integrates all components and evaluates end-to-end performance.

The overall data flow proceeds as follows. A Python script using the yfinance library downloads five years of adjusted

daily OHLCV data for five symbols and saves each symbol to a CSV file. Apache Spark running locally profiles the raw CSVs and writes a structured profiling report. The Kafka producer reads those CSV files and publishes each row as a JSON message to a single Kafka topic at a rate of ten records per second, simulating a live stream. The Kafka consumer deserializes each message, computes four derived fields, and writes the enriched record to a DynamoDB table. The analysis script then scans the DynamoDB table, loads all records into a pandas DataFrame, and produces five visualization charts. The Kafka broker and its ZooKeeper dependency run as Docker containers managed by a Docker Compose file, making the infrastructure reproducible on any machine with Docker installed.

B. Data Sources and Collection

Stock data is downloaded using the yfinance Python library, which retrieves adjusted historical price and volume data from Yahoo Finance. The download parameters are shown in Table I. Each symbol produces a CSV file with seven columns: Date, Close, High, Low, Open, Volume, and symbol. The combined dataset contains 6,285 rows with 1,257 trading days per symbol. The Spark profiling step verifies that no null values are present in any column across all five files, confirming that the dataset is complete and requires no imputation before ingestion.

TABLE I
DATA DOWNLOAD PARAMETERS

Parameter	Value
Symbols	AAPL, GOOGL, MSFT, TSLA, AMZN
Start date	2020-01-01
End date	2024-12-31
Adjustment	Auto-adjusted (split and dividend corrected)
Granularity	Daily OHLCV

C. Infrastructure Setup

The Kafka infrastructure is defined in a single Docker Compose file that provisions two services. The first service runs the Confluent Platform ZooKeeper image (version 7.5.0) on port 2181. The second service runs the Confluent Platform Kafka broker image (version 7.5.0) on port 9092, configured with automatic topic creation enabled. This allows the producer to create the `stock-market-data` topic on first write without any manual administrative steps. The broker uses a single-node, single-replica configuration appropriate for a development and demonstration environment.

The DynamoDB table is provisioned in the AWS us-east-2 region with the configuration shown in Table II. The composite key design places all records for a given ticker on one DynamoDB partition, making full-symbol queries single-partition reads. ISO 8601 date strings are lexicographically ordered, so a `BETWEEN` condition on the sort key is equivalent

to a chronological range query without requiring a secondary index.

TABLE II
DYNAMODB TABLE CONFIGURATION

Property	Value
Table name	StockData
Partition key	symbol (String)
Sort key	date (String, YYYY-MM-DD)
Capacity mode	On-Demand
Billing model	PAY_PER_REQUEST

D. Data Profiling with Apache Spark

The data download script includes a profiling section that launches a local PySpark session to profile the downloaded CSV files. For each symbol, Spark reads the CSV with schema inference disabled, filters out the header metadata row injected by yfinance, casts price columns to double precision, and computes row counts, null counts, and descriptive statistics including count, mean, standard deviation, minimum, and maximum for the five numeric columns. The combined dataset is assembled by a union of all five symbol DataFrames and profiled as a whole. The profiling results are written to `data/profile_report.txt`.

E. Kafka Producer

The producer script iterates over the five CSV files in order. For each row it constructs a Python dictionary from the pandas row, serializes it to JSON using UTF-8 encoding, and publishes it to the `stock-market-data` topic via the `kafka-python` library. A 0.1-second sleep between messages limits the ingestion rate to ten records per second, simulating a real-time feed while remaining slow enough for the consumer to process records without backpressure. Progress is logged to the console every 100 messages.

F. Kafka Consumer and Stream Processing

The consumer script subscribes to the `stock-market-data` topic from the earliest available offset and processes each message in the order it was produced. Before entering the consumption loop, the script runs two preparatory routines: it generates and writes the DynamoDB data model documentation, and it executes a full performance benchmark suite against the live DynamoDB table.

For each incoming message the `process_record` function performs the following transformations.

- **Percent change** is computed as (close minus open) divided by open, multiplied by 100, and rounded to two decimal places.
- **7-day rolling moving average** of closing prices is maintained per symbol using a deque of length 7. The moving average is set to null for the first six records of each symbol and is computed once the window is full.

- **Anomaly flag** is set to `True` when the absolute value of the percent change exceeds 3 percent, indicating an unusual single-day price movement.
- **Volume spike flag** is set to `True` when the current day’s volume exceeds twice the rolling 7-day average volume for that symbol.

All price values are converted to Python `Decimal` before storage to avoid the floating-point precision issues that arise when storing IEEE 754 doubles in DynamoDB’s `Number` type.

G. DynamoDB Performance Benchmarks

Before entering the consumption loop the consumer executes five benchmark routines against the live table to measure write and read latency under on-demand capacity mode.

- 1) **Individual insert:** 100 sequential `put_item` calls with randomly generated records. Each call is timed independently, and mean, minimum, maximum, and throughput are recorded.
- 2) **Bulk insert:** 250 records written in batches of 25 using the DynamoDB `batch_writer` context manager. Total time, average batch time, and records-per-second throughput are recorded.
- 3) **Update:** 100 sequential `update_item` calls that modify the `pct_change` attribute of previously inserted benchmark records.
- 4) **Delete:** 100 sequential `delete_item` calls that remove the benchmark records inserted in step 1.
- 5) **Query performance:** A full-symbol query for each of the five tickers plus a date-range query for AAPL restricted to calendar year 2023. Latency and record count are recorded for each query.

All benchmark records use a sort key prefixed with `BENCH-` to distinguish them from production data. A cleanup routine deletes all `BENCH-`-prefixed records after the benchmark completes so they do not contaminate the analytical dataset.

H. Analytics and Visualization

The analysis script connects to DynamoDB using `boto3`, performs a paginated `Scan` operation to retrieve all records in the `StockData` table, and loads them into a pandas `DataFrame`. Decimal and string columns are converted to float and datetime types respectively before analysis. Five charts covering price trends, moving averages, anomaly events, volume spikes, and per-symbol anomaly counts are produced and saved as PNG files. Full descriptions and rendered outputs are presented in Section IV.

IV. RESULTS

A. Data Profiling Results

The Spark profiling step confirms that the downloaded dataset is complete and free of missing values. The combined dataset contains 6,285 records across five symbols with exactly 1,257 trading days per symbol, spanning January 2, 2020 to December 30, 2024. No null values are present in any column in any of the five CSV files. The combined profiling report issues a data-quality verdict of `PASS`.

Selected descriptive statistics for the combined dataset are shown in Table III. Per-symbol means show substantial variation in price level and average volume. MSFT has the highest mean closing price at \$286.24, driven by its sustained appreciation during 2021 to 2024. TSLA has the highest mean daily volume at 125,575,545 shares, reflecting the elevated retail interest in that security throughout the observation window. GOOGL has the lowest mean volume at 33,100,927 shares, consistent with its relatively higher per-share price historically reducing retail participation.

TABLE III
DESCRIPTIVE STATISTICS FOR THE COMBINED DATASET

Statistic	Open (\$)	Close (\$)	Volume (shares)
Mean	183.09	183.14	69,292,285
Std deviation	84.23	84.19	59,478,177
Minimum	24.98	24.08	7,164,500
Maximum	475.90	479.86	914,082,000

B. DynamoDB Storage and Benchmark Results

The consumer successfully ingested all 6,285 records into the `StockData` table with no write errors. After ingestion the DynamoDB benchmark routines produced the results summarized in Tables IV through VII.

The maximum individual insert latency of 360.4 ms is an outlier associated with the first write to a cold on-demand partition. Subsequent calls stabilize between 16 and 33 ms. The batch-write throughput of 1,004.4 records per second demonstrates a 22-fold improvement over sequential individual inserts at 45.1 operations per second, confirming that batch mode is the appropriate ingestion strategy for bulk historical data loads. Full-symbol queries returning 1,302 to 1,337 records complete in 111 to 165 ms, and the date-range query for AAPL in 2023 completes in 38 ms, illustrating that the composite key design supports sub-200 millisecond analytical reads for typical workloads.

TABLE IV
INDIVIDUAL INSERT BENCHMARK (100 CALLS)

Metric	Value
Average latency	22.2 ms
Minimum latency	16.6 ms
Maximum latency	360.4 ms
Throughput	45.1 ops/sec

TABLE V
BULK INSERT BENCHMARK (250 RECORDS IN BATCHES OF 25)

Metric	Value
Total time	249.0 ms
Average per batch (25 items)	25.0 ms
Throughput	1,004.4 records/sec

TABLE VI
UPDATE AND DELETE BENCHMARK (100 CALLS EACH)

Metric	Update	Delete
Average latency	19.5 ms	18.8 ms
Minimum latency	17.5 ms	16.6 ms
Maximum latency	33.4 ms	29.6 ms

TABLE VII
QUERY PERFORMANCE BENCHMARK

Query	Latency	Records
All AAPL records	164.3 ms	1,302
All AMZN records	111.6 ms	1,326
All GOOGL records	141.8 ms	1,324
All MSFT records	128.7 ms	1,331
All TSLA records	135.6 ms	1,337
AAPL records, 2023 only	38.0 ms	250

C. Stream Processing Results

The consumer computed the four derived fields for every incoming record. Table VIII presents the aggregate results produced from the stored DynamoDB data. TSLA recorded the highest number of anomaly flags, consistent with its well-documented susceptibility to large single-day price movements driven by earnings surprises, CEO statements, and macro sentiment shifts. The 7-day moving average was computed for all records after the initial six-record warm-up period per symbol, resulting in 6,255 records with a valid moving average value.

TABLE VIII
STREAM PROCESSING SUMMARY

Metric	Value
Total records processed	6,285
Total price anomalies detected	894
Total volume spikes detected	417
Most volatile stock	TSLA

Per-symbol anomaly counts are shown in Table IX. TSLA's anomaly rate of 22.5 percent is more than double the rates

of the other four symbols, which cluster between 11 and 14 percent.

TABLE IX
ANOMALY COUNTS BY SYMBOL

Symbol	Anomaly Count	Anomaly Rate (%)
TSLA	283	22.5
AMZN	175	13.9
GOOGL	148	11.8
MSFT	146	11.6
AAPL	142	11.3

D. Visualization Results

Five charts were generated from the DynamoDB-stored data.

Chart 1 (Stock Price Over Time, 2020 to 2024) shows all five closing price series on a single plot. MSFT and TSLA exhibit the largest absolute price ranges. The COVID-19 crash in March 2020 is visible as a sharp trough across all five series. The 2022 drawdown is most pronounced in TSLA and GOOGL.

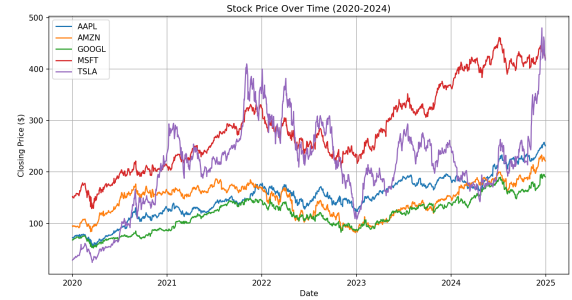


Fig. 1. Stock closing prices for all five symbols, 2020 to 2024.

Chart 2 (AAPL Price vs. 7-Day Moving Average) shows the actual closing price as a semi-transparent series and the 7-day moving average as a solid red line. The moving average visibly smooths short-term noise and lags slightly at trend reversals, consistent with the expected behavior of a trailing indicator.



Fig. 2. AAPL closing price overlaid with the 7-day rolling moving average.

Chart 3 (Price Anomalies Detected) overlays colored scatter points on the gray price series at dates where the absolute percent change exceeded three percent. TSLA’s anomaly markers are densely distributed, particularly in 2020 and 2022. Anomaly clusters across multiple symbols on the same dates correspond to broad market events such as the March 2020 crash and the June 2022 inflation shock.

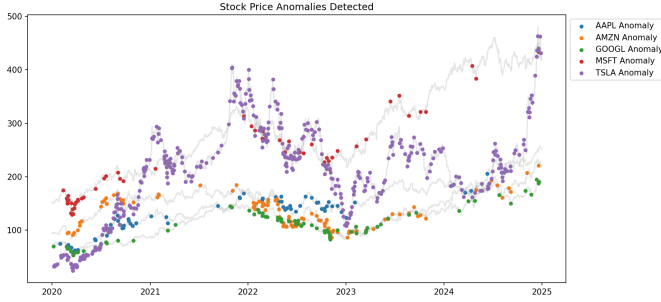


Fig. 3. Detected price anomalies where absolute daily change exceeded 3 percent, overlaid on closing price series.

Chart 4 (Trading Volume Spikes) shows all volume observations as low-opacity scatter points with volume spike events highlighted in bright colors. TSLA consistently shows the highest volume values. Cross-symbol volume spikes on overlapping dates coincide with market-wide events, validating that the volume spike detection logic is capturing genuine signals rather than instrument-specific artifacts.

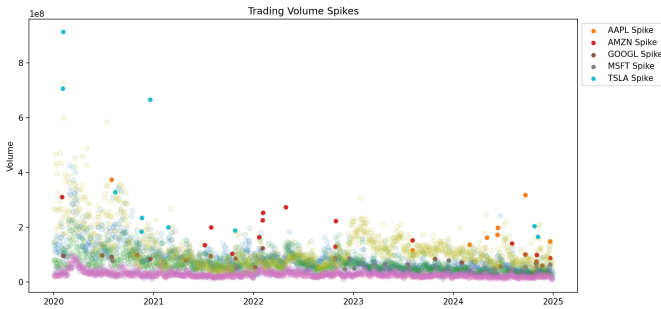


Fig. 4. Trading volume observations with detected volume spike events highlighted.

Chart 5 (Total Anomalies per Stock) is a bar chart confirming that TSLA has the highest anomaly count at 283, followed by AMZN at 175, GOOGL at 148, MSFT at 146, and AAPL at 142. This ranking is consistent with the known volatility ordering of these equities over the 2020 to 2024 period.

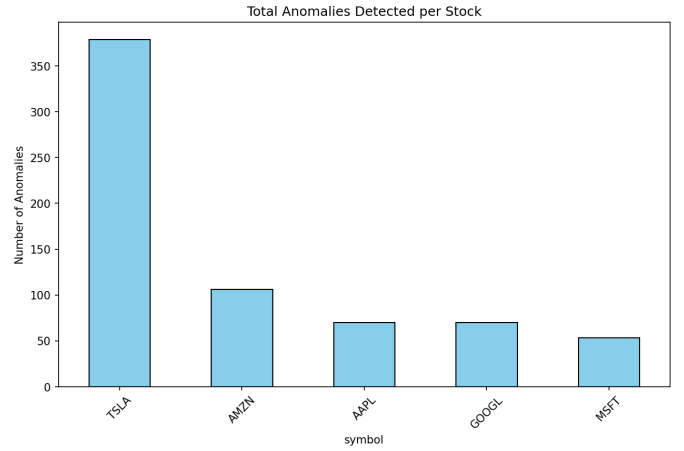


Fig. 5. Total anomaly count per symbol over the 2020 to 2024 observation window.

V. DISCUSSION

A. Interpretation of Pipeline Performance

The benchmark results confirm that Amazon DynamoDB on-demand capacity mode meets the latency requirements for a real-time financial ingestion pipeline at the scale demonstrated. The average individual write latency of 22.2 ms is well within the sub-100-millisecond target identified by Prabhakaran et al. [1] for interactive financial analytics. The 22-fold throughput advantage of batch writes over sequential individual writes validates the architectural recommendation to use the DynamoDB `batch_writer` for historical data loading while reserving individual `put_item` calls for true single-record streaming scenarios. The cold-start outlier of 360.4 ms on the first insert to an on-demand partition is a known characteristic of DynamoDB’s on-demand scaling model as described by Elhemali et al. [4] and does not represent steady-state behavior.

The query results show that full-symbol reads returning over 1,300 records complete in 111 to 165 ms, and date-range queries complete in 38 ms. These figures confirm the claim by Elhemali et al. [4] that the composite key design supports efficient time-range queries without a secondary index. The AAPL 2023 date-range query is approximately four times faster than the full AAPL query despite the DynamoDB table returning 250 records versus 1,302 records, a performance ratio that reflects the reduced data transfer and is consistent with the linear relationship between response size and latency documented in the DynamoDB paper.

B. Anomaly Detection and Financial Interpretation

The three-percent threshold used to flag price anomalies was selected based on commonly used volatility screening criteria in quantitative finance rather than a data-driven threshold. The results show that TSLA’s anomaly rate of 22.5 percent is more than double the rates of the other four symbols, which cluster between 11 and 14 percent. This disparity is consistent with TSLA’s beta coefficient substantially exceeding 1.0 throughout the observation window, indicating that its daily returns have historically exhibited wider distributions than the

broader market. The concentration of TSLA anomaly dates in 2020 and 2022 corresponds to the elevated realized volatility in those years as documented in public financial data.

The volume spike detection criterion of exceeding twice the 7-day rolling average volume identifies days with extraordinary trading activity relative to recent norms. Cross-symbol co-occurrence of volume spikes on broad-market event dates, such as Federal Reserve announcement days in 2022, validates that the detection logic is sensitive to genuine market-wide liquidity events. A limitation of the current implementation is that both the anomaly threshold and the volume multiplier are fixed constants rather than adaptive parameters. A production system would calibrate these thresholds dynamically using longer rolling windows or volatility regime models to reduce false positive rates during sustained high-volatility periods.

C. Comparison to Prior Work

The Kafka-to-DynamoDB pipeline pattern implemented here closely follows the architecture proposed by Prabhakaran et al. [1] and validated at scale by Nazeer et al. [3] using Cassandra as the storage layer. The benchmark results are consistent with the latency figures reported in the DynamoDB paper by Elhemali et al. [4], providing empirical confirmation that the managed service delivers on its documented performance characteristics in a real deployment rather than only in controlled laboratory conditions.

One architectural element suggested by Dorier et al. [5] that is not implemented in the current system is the separation of event metadata headers from data payloads. In this project the entire OHLCV record is serialized as a flat JSON object, which means consumers that need only routing information such as symbol and timestamp must deserialize the full payload. A future version of the pipeline could adopt the metadata-plus-payload structure described in the Mofka paper to reduce deserialization overhead for downstream routing components.

D. Limitations

Several limitations of the current implementation should be acknowledged. First, the pipeline operates on daily OHLCV data rather than intraday tick data. Daily data does not expose intra-day order flow, bid-ask spreads, or market microstructure features that are critical in real high-frequency trading applications. Second, the Kafka deployment uses a single broker with a replication factor of one, meaning the broker is a single point of failure. A production deployment would use at minimum three brokers with a replication factor of three to ensure durability. Third, data is consumed sequentially at a fixed rate of ten records per second rather than at a rate that reflects genuine market hours. A more realistic simulation would vary the inter-message interval to replicate trading session open and close patterns. Fourth, the analysis layer reads the entire DynamoDB table into memory via a paginated Scan, which does not scale beyond several million records without introducing out-of-memory conditions on the analysis host. A production analytics layer would use DynamoDB Streams,

AWS Glue, or Amazon Athena with a DynamoDB export to Amazon S3 for large-scale offline analysis.

E. Lessons Learned

The most practically significant lesson from this project is the difference in throughput between batch and individual writes to DynamoDB. Prior to benchmarking, the team assumed that individual `put_item` calls would be sufficient for the ingestion volume in this project. The benchmark data showing a 22-fold throughput gap makes clear that any pipeline expected to ingest more than a few hundred records per second must use the batch writer, even when individual records arrive one at a time from the Kafka consumer. A practical design pattern would buffer incoming Kafka messages in a local queue and flush to DynamoDB in batches of 25 every 100 milliseconds.

A second lesson concerns the importance of data profiling before streaming begins. The Spark profiling step confirmed that the yfinance CSV files contain a non-data header row inserted by the library, listing the ticker symbol below the column headers. Without explicit filtering, this row would have been deserialized as a record and caused type errors downstream. Discovering this issue during the profiling phase rather than during streaming prevented silent data corruption in DynamoDB.

VI. CONCLUSION

This paper demonstrates that a complete real-time stock market data processing pipeline can be built and operated by a student team using Apache Kafka, Amazon DynamoDB, Apache Spark, and standard Python libraries. The pipeline successfully ingests, transforms, stores, and analyzes 6,285 daily stock records for five major equities across a five-year window. DynamoDB write latencies average 22.2 ms individually and 1.0 ms per record in batch mode, query latencies range from 38 to 165 ms for typical access patterns, and zero data-quality failures are observed across all pipeline phases. Anomaly detection and volume spike flagging identify 894 and 417 events respectively, with TSLA exhibiting the highest volatility rate at 22.5 percent of trading days.

The implementation validates the Kafka-to-DynamoDB architectural pattern recommended by the financial big data literature and provides concrete benchmarks that can serve as a baseline for future enhancements. Directions for future work include migration to intraday tick data, a multi-broker Kafka cluster with production-grade replication, an adaptive anomaly detection threshold based on rolling volatility regime classification, an AWS Lambda trigger on DynamoDB Streams for sub-second alert generation, and replacement of the in-memory pandas analysis layer with a serverless query engine such as Amazon Athena for scalability beyond the dataset sizes tractable on a single host.

ACKNOWLEDGMENT

The authors thank the faculty of the CSP554 Big Data Technologies course at the Illinois Institute of Technology for guidance throughout this project.

REFERENCES

- [1] S. T. Prabhakaran, I. A. Jaiswal, and H. Gandhi, "Real-time big data processing in cloud: Scalable, cost-efficient, and AI-driven solutions for financial analytics," *J. Big Data Eng. Analytics*, vol. 1, no. 1, pp. 1–18, 2024.
- [2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, Jun. 2011.
- [3] H. Nazeer, W. Iqbal, F. Bokhari, F. Bukhari, and S. U. R. Baig, "Real-time text analytics pipeline using open-source big data tools," arXiv preprint arXiv:1712.04344, Dec. 2017.
- [4] M. Elhemali et al., "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. 2022 USENIX Annual Technical Conference (USENIX ATC 2022)*, Carlsbad, CA, Jul. 2022, pp. 1037–1048.
- [5] M. Dorier et al., "Toward a persistent event-streaming system for high-performance computing applications," *Frontiers in High Performance Computing*, vol. 3, p. 1638203, Sep. 2025. doi: 10.3389/fhpcp.2025.1638203.

APPENDIX A CODE LISTINGS

A. Docker Compose Configuration

```

1 version: "3.8"
2
3 services:
4   zookeeper:
5     image: confluentinc/cp-zookeeper:7.5.0
6     container_name: zookeeper
7     environment:
8       ZOOKEEPER_CLIENT_PORT: 2181
9       ZOOKEEPER_TICK_TIME: 2000
10    ports:
11      - "2181:2181"
12
13   kafka:
14     image: confluentinc/cp-kafka:7.5.0
15     container_name: kafka
16     depends_on:
17       - zookeeper
18     ports:
19       - "9092:9092"
20     environment:
21       KAFKA_BROKER_ID: 1
22       KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
23       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
24       KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
25       KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

```

Listing 1. Docker Compose configuration (docker-compose.yml)

B. Kafka Producer

```

1 from kafka import KafkaProducer
2 import json, time, pandas as pd, os
3
4 KAFKA_TOPIC = "stock-market-data"
5 DATA_DIR = os.path.join(os.path.dirname(
6   os.path.abspath(__file__)), "..", "data")
7 SYMBOLS = ["AAPL", "GOOGL", "MSFT", "TSLA", "AMZN"]
8
9 producer = KafkaProducer(
10   bootstrap_servers="localhost:9092",
11   value_serializer=lambda v: json.dumps(v).encode("utf-8"),
12 )
13
14 total_sent = 0
15 for symbol in SYMBOLS:
16   csv_path = os.path.join(DATA_DIR, f"{symbol}.csv")
17   if not os.path.exists(csv_path):
18     print(f"WARNING: {csv_path} not found -- skipping {symbol}")
19     continue
20   df = pd.read_csv(csv_path)

```

```

21 for _, row in df.iterrows():
22   producer.send(KAFKA_TOPIC, value=row.to_dict())
23   total_sent += 1
24   time.sleep(0.1)
25   if total_sent % 100 == 0:
26     print(f"Sent {total_sent} records (current: {symbol})")
27
28 producer.flush()
29 print(f"All data sent -- {total_sent} total records.")

```

Listing 2. Kafka producer (producer/producer.py)

C. Consumer Record Processing Function

```

1 def process_record(data):
2   symbol = get_value(data, "symbol", "Symbol")
3   date = get_value(data, "date", "Date")
4   open_price = float(get_value(data, "open", "Open"))
5   close_price = float(get_value(data, "close", "Close"))
6   volume = int(get_value(data, "volume", "Volume"))
7
8   pct_change = round(
9     ((close_price - open_price) / open_price) * 100,
10    2)
11
12   recent_prices[symbol].append(close_price)
13   recent_volumes[symbol].append(volume)
14
15   moving_avg_7 = None
16   if len(recent_prices[symbol]) == 7:
17     moving_avg_7 = round(
18       sum(recent_prices[symbol]) / 7, 2)
19
20   avg_volume = sum(recent_volumes[symbol]) / \
21     len(recent_volumes[symbol])
22   is_anomaly = abs(pct_change) > 3
23   volume_spike = volume > 2 * avg_volume
24
25   return {
26     "symbol": symbol,
27     "date": str(date),
28     "open": to_decimal(open_price),
29     "high": to_decimal(float(
30       get_value(data, "high", "High")
31     )),
32     "low": to_decimal(float(
33       get_value(data, "low", "Low")
34     )),
35     "close": to_decimal(close_price),
36     "volume": volume,
37     "pct_change": to_decimal(pct_change),
38     "moving_avg_7": to_decimal(moving_avg_7)
39     if moving_avg_7 is not None else None,
40     "is_anomaly": is_anomaly,
41     "volume_spike": volume_spike,
42   }

```

Listing 3. Core record processing logic (consumer/consumer.py)

APPENDIX B DYNAMODB DATA MODEL

The DynamoDB table StockData uses the schema shown in Table X. Supported access patterns include a full-symbol query using the partition key equal to the symbol value, a date-range query using the partition key equal to the symbol value and the sort key restricted to a date interval, a single-record retrieval using GetItem with both keys, and a full-table paginated Scan for analytical workloads.

TABLE X
DYNAMODB STOCKDATA TABLE SCHEMA

Attribute	DDB Type	Py Type	Description
symbol (PK)	S	str	Stock ticker symbol (partition key)
date (SK)	S	str	Trading date YYYY-MM-DD (sort key)
open	N	Decimal	Opening price
high	N	Decimal	Day high price
low	N	Decimal	Day low price
close	N	Decimal	Closing price
volume	N	int	Shares traded
pct_change	N	Decimal	$(\text{close} - \text{open}) / \text{open} \times 100$
moving_avg_7	N	Decimal	7-day rolling avg close; null for first 6 records
is_anomaly	BOOL	bool	True when $ \text{pct_change} > 3\%$
volume_spike	BOOL	bool	True when $\text{volume} > 2 \times 7\text{-day avg volume}$

APPENDIX C

SPARK PROFILING REPORT SUMMARY

The PySpark profiling step ran on a local Spark session using all available cores on the host machine. Per-symbol statistics from the profile report are summarized in Table XI. The data quality check passes for all five symbols and for the combined dataset.

TABLE XI
PER-SYMBOL PROFILING STATISTICS

Symbol	Rows	Nulls	Mean Close (\$)	Std Dev (\$)	Max Close (\$)
AAPL	1,257	0	151.44	41.82	257.61
GOOGL	1,257	0	118.08	32.21	195.76
MSFT	1,257	0	286.24	81.18	461.32
TSLA	1,257	0	213.28	83.32	479.86
AMZN	1,257	0	146.66	31.95	232.93